A popular way of running third-party services in a Kubernetes cluster is through Helm (https://helm.sh). Think of it as a package manager. To install software on your computer, you can use one of the operating system package managers, like Apt (Ubuntu), Homebrew (macOS), or Chocolatey (Windows); in Kubernetes, you can similarly use Helm, but we call them *charts* instead of *packages*.

At this stage in our cloud native journey, using Helm would be a bit premature and perhaps confusing. To fully understand how it works, it's essential to acquire more familiarity with Kubernetes first.

For the rest of the chapter, I'll assume you have a PostgreSQL instance running in your local cluster. If at any point you need to undeploy the database, you can run the `kubectl delete -f services` command from the same folder.

The following section will present the main Kubernetes concepts and guide you through deploying Spring Boot applications on your local cluster.

7.2 Kubernetes Deployments for Spring Boot

This section will walk you through the main Kubernetes objects you'll be working with as a developer and the vocabulary necessary to communicate efficiently with a platform team and deploy your applications to a cluster.

You have already gone through the containerization of a Spring Boot application. A Spring Boot application on Kubernetes is still packaged as a container, but it runs in a Pod controlled by a Deployment object.

Pods and Deployments are core concepts you need to understand when working with Kubernetes. Let's start by looking at some of their main characteristics, after which you'll practice declaring and creating Kubernetes resources to deploy the Catalog Service application.